

MLA0303-Reinforcement Learning

CO-3 AT-1

Coding Challenge

Name: S. Divya

Reg No: 192425163

1. RL Framework: Agent–Environment Interaction

1. Concept and Core Explanation

Reinforcement Learning (RL) is a machine learning approach in which an agent learns sequential decision-making by interacting with an environment. At every time step, the agent observes a state, selects an action according to a policy, receives a reward, and reaches a new state. Unlike supervised learning, the correct action is not directly provided; the agent discovers useful behavior through experience.

The three essential elements are state, action and reward. A state represents the current situation available to the agent. An action is a decision that the agent can take. A reward is numerical feedback indicating whether the outcome was desirable. Positive rewards encourage useful behavior, while penalties discourage undesirable behavior.

2. Technical Explanation

The interaction is:

State $S_t \rightarrow$ Agent observes $S_t \rightarrow$ Policy selects $A_t \rightarrow$ Environment responds $\rightarrow$ Reward R_{t+1} + next state $S_{t+1} \rightarrow$ Learning update $\rightarrow$ Repeat.

The objective is to maximize cumulative discounted reward:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

The discount factor γ determines the importance of future rewards. Therefore, the best action is not necessarily the action with the highest immediate reward.

3. Example / Illustration

For example, in an autonomous delivery robot, the state may contain the robot location, obstacle distances, battery level and destination. Actions may be move forward, turn left, turn right or recharge.

Reaching the destination receives a positive reward, while collisions, unnecessary movement and battery depletion receive penalties.

The robot learns from repeated episodes. If a particular route repeatedly produces high cumulative reward, the policy becomes more likely to select that route in similar states.

4. Analytical Discussion

Cumulative reward is important because decisions have consequences over several time steps. A small immediate sacrifice may produce a much larger future benefit. For example, a robot may temporarily move away from the destination to avoid an obstacle and reach the target safely.

Reward design must be aligned with the actual objective. If only speed is rewarded, the robot may take unsafe routes. Adding safety and energy penalties creates a more meaningful objective.

5. Evaluation and Practical Relevance

Performance can be evaluated using average cumulative reward, success rate, number of steps, energy consumption, collision rate and convergence of the learning curve.

Thus, agent–environment interaction provides the foundation of RL. States describe situations, actions represent choices, rewards provide feedback, and cumulative reward guides the agent toward policies that produce better long-term outcomes.

6. Final Conclusion

This answer directly addresses the question requirement and demonstrates accurate technical knowledge, logical problem-solving, analytical thinking, professional communication and the ability to respond with relevant examples.

Rubric focus: Technical Knowledge (25%) • Problem Solving (25%) • Analytical Thinking (15%) • Communication (15%) • Confidence (15%) • Response to Questions (10%)

2. Markov Decision Process (MDP)

1. Concept and Core Explanation

A Markov Decision Process is a mathematical model for sequential decision-making under uncertainty. An MDP is represented by (S, A, P, R, γ) : state space S , action space A , transition probability P , reward function R and discount factor γ .

The Markov property states that the future depends on the current state and action rather than the complete history. This makes it possible to calculate values and derive optimal policies.

2. Technical Explanation

The state space contains all possible situations. The action space contains all legal decisions. The transition probability $P(s'|s,a)$ describes the probability of moving to state s' after taking action a in state s .

The reward function $R(s,a,s')$ gives feedback for a transition. The discount factor γ controls the importance of future rewards. A low γ emphasizes immediate rewards, while a value near 1 gives stronger importance to long-term consequences.

3. Example / Illustration

For example, in a warehouse robot, a state may represent the robot's position and nearby obstacles. Actions can be move forward, left, right or stop. Because sensors can be noisy, an action may lead to several possible next states with different probabilities.

The MDP can therefore be represented as:

Current state $\rightarrow$ Action $\rightarrow$ Probabilistic transition $\rightarrow$ Reward $\rightarrow$ Next state.

4. Analytical Discussion

The optimal policy π^* is the policy that maximizes expected cumulative discounted reward. Dynamic programming uses the Bellman equations to evaluate states and improve decisions.

$$V^*(s) = \max_a \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V^*(s')].$$

This equation states that the value of a state is the best expected immediate reward plus discounted future value.

5. Evaluation and Practical Relevance

Clearly defining MDP components converts a real-world problem into a mathematical decision model. Incorrect state information, missing actions or poorly designed rewards can produce poor policies. Therefore, MDP formulation is a critical step before applying policy iteration, value iteration or other RL algorithms.

6. Final Conclusion

This answer directly addresses the question requirement and demonstrates accurate technical knowledge, logical problem-solving, analytical thinking, professional communication and the ability to respond with relevant examples.

Rubric focus: Technical Knowledge (25%) • Problem Solving (25%) • Analytical Thinking (15%) • Communication (15%) • Confidence (15%) • Response to Questions (10%)

3. Policy and Value Functions with Bellman Equation

1. Concept and Core Explanation

A policy describes how an agent selects actions. A deterministic policy chooses one action for each state, while a stochastic policy assigns probabilities to actions. Value functions evaluate the quality of states and actions so that the agent can make decisions based on long-term benefits.

The state-value function $V_{\pi}(s)$ measures expected cumulative reward from state s under policy π . The action-value function $Q_{\pi}(s,a)$ measures expected cumulative reward when action a is taken in state s and the policy is followed afterward.

2. Technical Explanation

The state-value function is:

$$V_{\pi}(s) = E_{\pi}[G_t | S_t=s].$$

The action-value function is:

$$Q_{\pi}(s,a) = E_{\pi}[G_t | S_t=s, A_t=a].$$

The Bellman expectation equation is:

$$V_{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V_{\pi}(s')].$$

For optimal control:

$$V^*(s) = \max_a \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V^*(s')].$$

3. Example / Illustration

Consider a robot with two possible routes. One route gives a high immediate reward but leads to a difficult future state. The second route gives a smaller immediate reward but reaches a highly useful future state. The value function captures the future benefit, so the second route may receive a higher overall value.

Q-values are especially useful when the agent must select directly among actions.

4. Analytical Discussion

Policy evaluation estimates how good the current policy is. Policy improvement then chooses actions that have higher value. Repeating these two stages forms policy iteration. Value iteration directly applies the Bellman optimality update to improve value estimates.

This demonstrates the relationship:

Policy → Value evaluation → Better action selection → Improved policy.

5. Evaluation and Practical Relevance

Value functions allow RL systems to reason beyond immediate rewards. The Bellman equation provides a recursive method for calculating long-term value. Therefore, policy, state-value and action-value functions form the mathematical foundation for many dynamic programming and RL methods.

6. Final Conclusion

This answer directly addresses the question requirement and demonstrates accurate technical knowledge, logical problem-solving, analytical thinking, professional communication and the ability to respond with relevant examples.

Rubric focus: Technical Knowledge (25%) • Problem Solving (25%) • Analytical Thinking (15%) • Communication (15%) • Confidence (15%) • Response to Questions (10%)

4. Exploration–Exploitation: ϵ -Greedy and Softmax

1. Concept and Core Explanation

Exploration means trying uncertain actions to gain information. Exploitation means selecting the action currently believed to have the highest reward. Both are necessary: pure exploitation can lock the agent into a suboptimal policy, while excessive exploration wastes experience.

The balance is especially important during early learning when value estimates are uncertain.

2. Technical Explanation

In ϵ -greedy exploration, the agent selects a random action with probability ϵ and the currently best action with probability $1-\epsilon$. A common approach is to start with a larger ϵ and gradually reduce it.

For example, with $\epsilon=0.1$, approximately 10% of decisions explore and 90% exploit. The strategy is simple and works well for many discrete-action problems.

3. Example / Illustration

Softmax exploration selects actions probabilistically according to their estimated values:

$$P(a|s) = \exp(Q(s,a)/\tau) / \sum_b \exp(Q(s,b)/\tau).$$

The temperature τ controls exploration. High temperature makes action probabilities more equal; low temperature makes the highest-valued action dominant.

4. Analytical Discussion

ϵ -greedy treats exploratory actions approximately equally, whereas softmax gives higher probability to actions whose values are closer to the best action. ϵ -greedy is easier to implement, while softmax provides more informed exploration.

In safety-critical systems, exploration should be restricted or carried out in simulation because random physical actions can be dangerous.

5. Evaluation and Practical Relevance

Therefore, exploration discovers potentially better actions and exploitation uses learned knowledge. ϵ -greedy is simple and robust, while softmax provides value-sensitive exploration. Choosing an appropriate strategy improves learning efficiency and reduces premature convergence.

6. Final Conclusion

This answer directly addresses the question requirement and demonstrates accurate technical knowledge, logical problem-solving, analytical thinking, professional communication and the ability to respond with relevant examples.

Rubric focus: Technical Knowledge (25%) • Problem Solving (25%) • Analytical Thinking (15%) • Communication (15%) • Confidence (15%) • Response to Questions (10%)

5. Reward Shaping

1. Concept and Core Explanation

Reward shaping adds useful intermediate feedback to an RL problem when the original reward is sparse or delayed. It can make learning faster by indicating progress toward the final objective.

For example, a robot that receives reward only after reaching a destination may spend many episodes receiving no useful signal. A small progress-based reward can guide it toward the target.

2. Technical Explanation

A shaped reward can be written as:

$$R' = R_{\text{task}} + F(s, s'),$$

where R_{task} is the original task reward and F is an additional guidance signal.

A navigation task may reward reduction in distance to the goal, while penalizing collision, energy consumption and unnecessary movement.

3. Example / Illustration

Reward shaping accelerates learning by improving feedback and credit assignment. However, poor shaping can create unintended behavior. If the agent is rewarded only for moving closer to the target, it may exploit the reward without actually completing the task.

Therefore, the shaping signal must remain aligned with the real objective.

4. Analytical Discussion

Potential-based shaping is useful because, under appropriate conditions, it can improve learning without changing the optimal policy. Reward design should also include penalties for unsafe or undesirable behavior.

Learning curves, success rate, cumulative reward and observed policy behavior should be checked for unintended strategies.

5. Evaluation and Practical Relevance

Reward shaping can substantially improve sample efficiency and learning speed. It should provide guidance without replacing the true task objective. A carefully designed reward helps the agent avoid poor local strategies and discover useful policies faster.

6. Final Conclusion

This answer directly addresses the question requirement and demonstrates accurate technical knowledge, logical problem-solving, analytical thinking, professional communication and the ability to respond with relevant examples.

Rubric focus: Technical Knowledge (25%) • Problem Solving (25%) • Analytical Thinking (15%) • Communication (15%) • Confidence (15%) • Response to Questions (10%)

6. RL in Real-World Robotics

1. Concept and Core Explanation

Physical robotics presents challenges that are much harder than simulation. The agent must learn while operating equipment that can be damaged. The three major concerns are safety, sample efficiency and environment variability.

Safety means preventing collisions, excessive force and dangerous actions. A learning algorithm cannot simply experiment without restrictions on a real robot.

2. Technical Explanation

Sample efficiency describes how much experience is required to learn. Physical experiments are expensive and slow compared with simulation. Simulation-to-real transfer, offline datasets and model-based planning can reduce the number of physical trials.

Environment variability includes lighting changes, sensor noise, surface friction, payload differences, moving people and unexpected obstacles.

3. Example / Illustration

A practical workflow is:

Simulation → Safe exploration → Policy training → Hardware testing → Restricted deployment → Monitoring → Improvement.

Domain randomization can vary simulated conditions so that the learned policy becomes more robust to real-world differences. Safety controllers can override unsafe RL actions.

4. Analytical Discussion

Evaluation should consider task success, collision rate, training samples, energy consumption, recovery ability and robustness to changing conditions. Human supervision is important for high-risk applications.

A fallback controller can return the robot to a safe state when the RL policy becomes uncertain or violates safety limits.

5. Evaluation and Practical Relevance

RL is powerful for robotics because it can learn complex control strategies, but unrestricted trial-and-error is unsuitable for real machines. Combining simulation, efficient learning, safety constraints and robust control provides a safer and more practical solution.

6. Final Conclusion

This answer directly addresses the question requirement and demonstrates accurate technical knowledge, logical problem-solving, analytical thinking, professional communication and the ability to respond with relevant examples.

Rubric focus: Technical Knowledge (25%) • Problem Solving (25%) • Analytical Thinking (15%) • Communication (15%) • Confidence (15%) • Response to Questions (10%)

7. Discount Factor γ

1. Concept and Core Explanation

The discount factor γ determines how strongly future rewards influence present decisions. The return is:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

A small γ makes the agent short-sighted, while a larger γ makes it more long-term oriented.

2. Technical Explanation

Suppose Action A gives an immediate reward of 8. Action B gives 2 immediately and 15 in the next step. With a low γ , Action A may be preferred. With a high γ , the future reward from Action B becomes more important.

Thus, γ changes the effective planning horizon.

3. Example / Illustration

In episodic tasks, the episode eventually terminates. Terminal rewards can represent success or failure. A high γ allows the agent to value rewards that occur many steps before the terminal result.

For example, in a game, an agent may sacrifice a small immediate reward to increase the probability of winning at the end of the episode.

4. Analytical Discussion

Choosing γ requires considering the task. Fast control tasks may use a lower value, while strategic planning and resource management often require a higher value. A very high γ can increase sensitivity to uncertain distant outcomes, while a very low γ can produce myopic behavior.

5. Evaluation and Practical Relevance

Therefore, γ controls the balance between immediate and long-term decision-making. In episodic tasks, it determines how strongly terminal and delayed rewards affect current actions. Correct selection of γ is essential for learning a suitable policy.

6. Final Conclusion

This answer directly addresses the question requirement and demonstrates accurate technical knowledge, logical problem-solving, analytical thinking, professional communication and the ability to respond with relevant examples.

Rubric focus: Technical Knowledge (25%) • Problem Solving (25%) • Analytical Thinking (15%) • Communication (15%) • Confidence (15%) • Response to Questions (10%)

8. Model-Free vs Model-Based RL

1. Concept and Core Explanation

Model-free RL learns policies or value functions directly from experience without explicitly learning the environment dynamics. Examples include Q-learning and policy-gradient methods.

Model-based RL uses or learns a model of the environment and then uses that model for planning. The main difference is whether an explicit transition model is available for prediction.

2. Technical Explanation

Model-free methods are simpler and avoid errors caused by an inaccurate learned model. They can perform well when large amounts of experience are available. Their main limitation is sample inefficiency.

Model-based methods can plan possible future trajectories and reuse experience through simulated rollouts. This can improve sample efficiency, especially when real-world interactions are expensive.

3. Example / Illustration

In dynamic environments, model-based RL can update its model as conditions change. However, model errors can lead to incorrect plans. Model-free methods avoid explicit model error but may require many interactions before adapting to a changed environment.

4. Analytical Discussion

A hybrid design can combine both approaches: a learned model can support planning while a model-free policy handles complex behavior. Evaluation should consider sample efficiency, adaptation speed, computational cost, robustness and final reward.

5. Evaluation and Practical Relevance

Neither approach is universally superior. Model-based RL is attractive when planning and sample efficiency are important, while model-free RL is useful when learning directly from experience is easier than learning accurate dynamics. The environment and data constraints should determine the choice.

6. Final Conclusion

This answer directly addresses the question requirement and demonstrates accurate technical knowledge, logical problem-solving, analytical thinking, professional communication and the ability to respond with relevant examples.

Rubric focus: Technical Knowledge (25%) • Problem Solving (25%) • Analytical Thinking (15%) • Communication (15%) • Confidence (15%) • Response to Questions (10%)

9. Q-Learning Update and Convergence

1. Concept and Core Explanation

Q-learning is a model-free, off-policy RL algorithm that learns the optimal action-value function. It does not require transition probabilities. The algorithm repeatedly updates Q-values using observed rewards and the best estimated value of the next state.

The update is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)].$$

2. Technical Explanation

Consider $Q(S1, \text{Right})=0$. The agent chooses Right, receives reward 5 and reaches S2. Suppose $\max_{a'} Q(S2,a')=4$, $\alpha=0.5$ and $\gamma=0.9$.

$$\text{Target} = 5 + 0.9(4) = 8.6.$$

$$\text{New } Q(S1, \text{Right}) = 0 + 0.5(8.6 - 0) = 4.3.$$

The estimate moves toward the observed target rather than changing completely.

3. Example / Illustration

If a later target becomes 10:

$$\text{New } Q = 4.3 + 0.5(10 - 4.3) = 7.15.$$

Repeated observations gradually refine the estimate. The process is called temporal-difference learning because the current estimate is updated using a target involving the next state's estimate.

4. Analytical Discussion

The training process is:

Initialize Q → Observe state → Choose action → Receive reward and next state → Calculate TD target → Update Q → Repeat.

With sufficient exploration and appropriate learning conditions in a stationary finite MDP, Q-learning can converge toward optimal action values. Learning curves and stabilization of Q-values can be used to examine convergence.

5. Evaluation and Practical Relevance

Q-learning demonstrates how simple repeated updates can transform experience into useful long-term action estimates. It is widely important because it can learn optimal behavior without a known environment model.

6. Final Conclusion

This answer directly addresses the question requirement and demonstrates accurate technical knowledge, logical problem-solving, analytical thinking, professional communication and the ability to respond with relevant examples.

Rubric focus: Technical Knowledge (25%) • Problem Solving (25%) • Analytical Thinking (15%) • Communication (15%) • Confidence (15%) • Response to Questions (10%)

10. On-Policy vs Off-Policy Learning

1. Concept and Core Explanation

On-policy learning improves the policy that is actually being used to generate experience. SARSA is a standard example. Off-policy learning separates the behavior policy from the target policy. Q-learning is a standard example.

This distinction is important because exploration and the policy being learned do not have to be identical in off-policy methods.

2. Technical Explanation

SARSA updates using the action actually selected next:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma Q(s',a') - Q(s,a)].$$

Q-learning updates toward the best next action:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)].$$

Therefore, SARSA learns the policy being followed, whereas Q-learning learns toward a greedy optimal target.

3. Example / Illustration

Advantages of on-policy learning include awareness of actual exploratory behavior and potentially safer or more conservative behavior. SARSA can account for the consequences of exploration.

Limitations include dependence on the behavior policy and potentially slower learning of a purely greedy optimal policy.

4. Analytical Discussion

Off-policy methods can learn an optimal target policy while following a different behavior policy. They can also reuse historical data and experience replay. However, the mismatch between behavior and target policies can create instability in some settings.

For safety-critical applications, unrestricted exploration should be avoided; constrained or offline learning may be preferable.

5. Evaluation and Practical Relevance

In conclusion, on-policy methods learn the policy used for behavior, while off-policy methods can learn a different target policy. On-policy learning is useful when actual behavior matters, whereas off-policy learning offers flexibility and data reuse. The choice depends on safety, exploration, data availability and environment dynamics.

6. Final Conclusion

This answer directly addresses the question requirement and demonstrates accurate technical knowledge, logical problem-solving, analytical thinking, professional communication and the ability to respond with relevant examples.

Rubric focus: Technical Knowledge (25%) • Problem Solving (25%) • Analytical Thinking (15%) • Communication (15%) • Confidence (15%) • Response to Questions (10%)